

A Service Selection Mechanism using Fault-Tolerance Techniques

Higor Amario de Souza¹, Felipe Pontes Guimaraes¹, Fabio Kon¹, Daniel Macedo Batista¹

¹Department of Computer Science – IME
University of Sao Paulo (USP) – Sao Paulo, SP, Brazil

{hamario, felipepg, kon, batista}@ime.usp.br

Abstract. *Service selection in Service-oriented computing (SOC) is an important step towards obtaining the required level of quality. Fault-Tolerance Techniques (FTT) can be used to enhance the reliability of compositions considering Quality of Service (QoS) requirements. Services that do not perform as expected must be replaced by equivalent services. We propose a mechanism called FTTSERVICESELECTOR (FSS) to select services that meet QoS constraint values defined by the user. FSS uses a Constraint-Satisfaction Problems (CSP) approach to generate a global optimal service selection. Experiments show that FSS can be used to generate solutions to form compositions with up to ten roles and ten equivalent services. In most cases, FSS can generate solutions in a feasible time.*

1. Introduction

Service-oriented computing (SOC) is the computing paradigm that introduced the concept of software-as-a-service. Services are self-describing, platform-agnostic computational elements that can be used to compose distributed applications [Papazoglou 2003]. Services can also be viewed as functions accessible via well-established protocols. These services can vary from small to complex functions. Services distributed over the Internet are known as Web services. They can be used by organizations in their business processes in a wide range of devices, such as desktops and smartphones [Papazoglou 2003]. Web services are broadly used for several activities, from obtaining information about weather to performing complex stock exchange transactions.

According to Papazoglou [Papazoglou 2003], the main characteristics of services to be offered to organizations over the Internet are: technology neutrality, loose coupling and location transparency. *Technology neutrality* relies on compatibility with almost all computational environments. *Loose coupling* is related to independence of knowledge about internal structures or conventions between clients and servers. *Location transparency* is concerned with the availability of the services.

Service-oriented architecture (SOA) is a way of reorganizing software applications and support infrastructure to provide an interconnected set of services to perform SOC. It is an architecture composed of service providers, service discovery agencies and service requestors [Papazoglou 2003]. In SOA, the communication between clients and services occurs using repositories that keep information about available services. Well-defined protocols are used to exchange requests and responses.

Applications exposed as services can be combined to implement standard business processes. Service composition is a term used to define applications that use

different services to produce high level functionalities. Service compositions can lead to the development of rapid, low-cost and interoperable distributed applications [Papazoglou et al. 2007]. Orchestration and choreography are the main models of service compositions to create business processes. Orchestration refers to an executable business process that interacts with services. Choreography refers to compositions in which the control of the interactions is decentralized – the services involved are only aware of their own roles in the interactions [Peltz 2003]. In a pool of services it is possible that there are equivalent services – if a service is not available it may be replaced by some of its equivalent services so compositions do not suffer from unexpected failures.

Although compositions have several benefits, there are some challenges to improve their delivered QoS. The number of available services has increased over the years [Rao and Su 2005]. As a consequence, there is an increasing number of selectable services to form compositions. Services could be unavailable or updated on the fly. Thus, selecting services that meet certain quality criteria is an important task to obtain reliable compositions.

This paper proposes a mechanism to select services for compositions from combinations of services that are generated using four Fault-Tolerance Techniques (FTT): *Retry*, *Recovery Block*, *N-Version Programming*, and *Active Replication*. We use a Constraint-Satisfaction Problem (CSP) approach to generate these combinations of services. Combinations of services that meet specific QoS constraint values are selected. The QoS requirements used were *Response Time* and *Reliability*. This information can be used by decision models to choose services more suitable to the constraints.

The remainder of this paper is organized as follows. Section 2 shows the key concepts used in this work. Section 3 presents related work. Section 4 presents the proposed FSS mechanism and its related concepts. Section 5 presents our experimental evaluation and the results are shown in Section 6. Then, in Section 7 we discuss the results. Section 8 shows the threats to validity and we present the conclusions and future work in Section 9.

2. Key Concepts

The FSS mechanism uses CSP to generate all possible combinations of compositions based on FTT. Then, QoS requirements are used to the service selection process. These key concepts are presented in this section.

2.1. Quality of Service (QoS)

In distributed systems, QoS is a combination of qualities or properties of a service [Menasce 2002]. Some qualities measured in distributed systems are availability, cost, response time, and reliability. *Availability* is the percentage of time that a service is operating. *Cost* is the effort spent by the service. *Response time* is the time spent by the service to respond a request. *Reliability* is related to the correctness of the response.

QoS requirements can be used to enhance the quality of applications according to desired criteria. Thus, the reliability of compositions can be improved using QoS requirements to select services so that the formed composition keeps certain quality requirements. These values can be seen as constraints chosen by an interested user.

2.2. Fault-Tolerance Techniques (FTT)

Fault-tolerance is the ability to deliver a service that can be trusted, avoiding service failures in the presence of faults [Avizienis et al. 2004]. Fault-tolerance techniques (FTT) are used in different areas of computer systems: embedded systems, microprocessors, or grid applications. Considering that any system is failure prone, fault-tolerance techniques can help to avoid the loss of program executions, loss of data, or even to finish the execution of a function without break a software.

FTT can be used to enhance the reliability of a composition operation in the presence of faults. Once we have equivalent services to choose for compositions, FTTs can be used to generate different combinations of these services. Thus, it is possible to choose combinations of services more suitable to the composition execution according to the expected input constraint values.

Some FTTs used in distributed systems came from other computing areas. These FTTs can be divided as sequential and parallel approaches. Two sequential techniques are Recovery Block and Retry. *Recovery Block* (RB) was first used for error recovery in exception handling of software systems. The RB approach aims at providing fault-tolerant functional components that may be nested within a sequential program [Randell and Xu 1995]. Should some component fail during execution, it is replaced by an alternative component to continue the sequence of the execution. *Retry* is a variation of RB that uses only one component for each functionality in a sequential program. This component is tried repeatedly until it violates a deadline or else it produces an expected output [Ammann and Knight 1988].

Two parallel techniques are N-Version Programming (NVP) and Active Replication (Active). *NVP* is a technique that uses the results from different program versions to choose a consensual result [Avizienis 1995]. The idea is that different versions can reduce the probability of obtaining an erroneous result. *Active replication* invokes software component equivalent services on distinct host computers concurrently [Chereque et al. 1992].

In the distributed systems domain, FTTs can be used to invoke equivalent services in compositions. These techniques can inhibit failures of a service in a composition, replacing it by other and keep the application working correctly. However, it is necessary to choose which FTT to use and what are the best services to meet the expected composition behavior. Given a set of services, the feasibility of combining these services to form compositions should be evaluated.

2.3. Constraint-Satisfaction Problems (CSP)

A large number of problems in different computer science areas can be viewed as Constraint-Satisfaction Problems (CSP) such as diagnostic reasoning, scheduling, or decision process [Kumar 1992]. A CSP can be defined as a problem with a given set of variables, a finite and discrete domain for each variable, and a set of constraints. The goal is to find possible solutions that satisfy these constraints [Kumar 1992].

There are different approaches to solve CSP. Algorithms to solve CSP usually fall into two main families: systematic and local search algorithms [Jussien and Lhomme 2002]. Two examples of systematic and local search algorithms

are, respectively, Backtracking and Constraint Propagation. *Backtracking* is a systematic algorithm that uses all possible combinations between the variables to test which of them satisfy the constraints [Kumar 1992]. *Constraint Propagation* is a local search algorithm. It considers that information is deduced from a local group of constraints and nodes, and is recorded as a change in the network. Further deductions make use of these changes to make further changes [Davis 1987].

Backtracking could be used to generate all potential solutions for CSPs. However, the backtracking runtime complexity is exponential [Kumar 1992]. Conversely, local search algorithms such as Constraint Propagation can reduce the complexity, but they may not find a solution [Jussien and Lhomme 2002].

The problem of selecting services for compositions that meet QoS constraints can be viewed as a CSP. In this work, we use the backtracking approach in the FTTSERVICE-Selector mechanism to do this task, since we are interested in a method that returns a solution. The QoS requirements are the constraints of our problem. Each FTT can generate a different number of combinations of equivalent services that could lead to possible solutions to these constraints. Thus, it is possible to use CSP to generate solutions in a feasible time using these QoS constraints for compositions with up to 10 roles and 10 equivalent services per role.

3. Related Work

There are other techniques that use QoS requirements to the service selection task. Katterpur et al. [Katterpur et al. 2013] investigate the use of QoS in choreographies. In this work, they measured QoS requirements such as *Deadlock Freeness*, *Conformance* and *Realizability*. All these parameters are related to the messages passed among the services. Yu and Lin [Yu and Lin 2005] propose a technique to use QoS to select services for compositions. They use the concept of QoS broker to maximize the utility function value.

Ai and Tang [Ai and Tang 2008] present a genetic algorithm that uses a penalty-based concept to the service selection. The results show that the technique is scalable for compositions with up to 100 roles. However, the technique finds local optimal solutions. Alrifai et al. [Alrifai et al. 2012] propose a technique that combines local and global optimal selection. They used mixed integer programming to find the global optimization of QoS requirements as local constraints and local optimization to select the services that matches these local constraints. The evaluation carried out using simulated datasets show that the technique can reduce the time spent to find solutions.

Other previous works combine the use FTTs and QoS to select services for compositions dynamically. Zheng and Lyu [Zheng and Lyu 2010] propose a middleware that uses QoS requirements with different FTTs. The QoS constraints are informed by users. The authors combine the four FTTs summarized in Section 1 to propose a FTT called *Dynamic*. The experiments show that Dynamic outperforms the other FTTs used separately.

Dobson [Dobson 2006] uses WS-BPEL to model fault-tolerance strategies for processes. These models include FTTs such as the four mentioned in the previous section. A tool for mapping FTTs in WS-BPEL processes is presented. Preliminary experiments are carried out to show the feasibility of the approach. Salatge and Fabre

[Salatge and Fabre 2007] propose the idea of connectors to deal with unreliable services. These connectors are modeled using WSDL to insert detection actions such as replication strategies in the presence of failures. Thus, equivalent services can be defined to replace unreliable services for compositions. The authors carried a case study simulating equivalent services to show the feasibility of their approach.

Differently from the previous works, our mechanism can be used to construct combinations of services for compositions before their execution. Moreover, FSS can collect QoS values from previous executions to the service selection process that can be used in future executions. We use the CSP approach to evaluate all possible compositions that meet the expected QoS constraints. These constraints can be defined to keep the desired QoS values for compositions. The results can be used by other techniques to take decisions about what are the best solutions to use when enacting a composition.

4. FTServiceSelector (FSS)

FTServiceSelector (FSS) is based on the CSP backtracking approach. The problem solved by FSS is related to generating all possible combinations between roles and equivalent services that could be combined according to the FTT strategies. These combinations are evaluated to select services that meet the given QoS constraints. Thus, these services can be used in compositions. We use two QoS requirements in FSS: *response time* and *reliability* for the complete composition as the requirements to select suitable combinations of services.

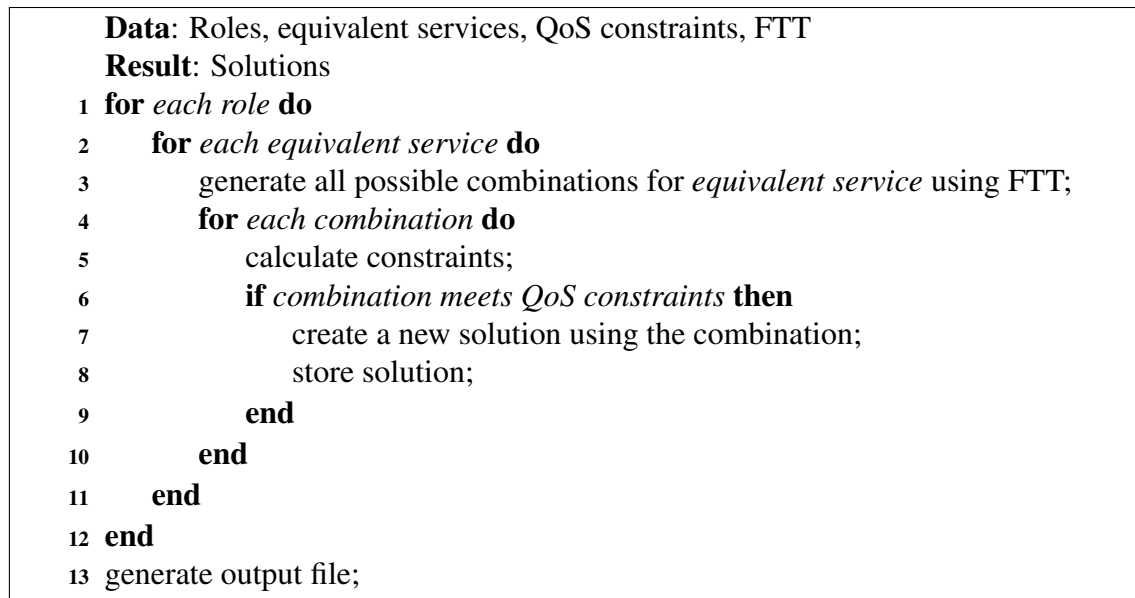
Algorithm 1 presents the FSS mechanism. It generates all possible combinations of services according to the characteristics of the FTTs. This algorithm runs once for each FTT separately. The four FTTs are also considered as variables of our problem: *Retry* requests the same Web service a certain number of times; *Recovery Block* (RB) requests equivalent services if a primary Web service fails; *N-Version Programming* invokes equivalent services at the same time and chooses the consensual result; *Active Replication* invokes equivalent services and takes the first returned result. Each of these techniques leads to different ways to calculate response time and reliability (QoS constraints) of compositions using the response time and the reliability of each service involved. The combinations that meet the QoS constraints are stored as solutions.

FSS receives input files in yaml¹ format. These files represent the resources available: a list of roles and the equivalent services with their response time and reliability values. FSS also receives an input file with the expected QoS constraint values for the whole composition. Then, FSS generates the combinations of services according to the FTT selected using the backtracking strategy (lines 1-3). Next, FSS uses the formulas presented in the Subsection 4.1 to calculate the constraints for each combination (lines 4-5) and store the combinations that meet the QoS constraints (lines 6-9) as solutions. The output is a yaml file that contains all solutions for each FTT used (line 13). To reduce the memory space used, FSS calculates each combination and records only those that satisfy the QoS values.

Figure 1 shows the FSS process. The mechanism receives files containing roles, available services, and QoS constraint values. Then, FSS generates all possible combinations of services based on FTTs. If the values of a combination meet the QoS constraint

¹<http://www.yaml.org/spec/1.2/spec.html>

values, this combination is recorded as a solution. After obtaining all solutions, FSS creates an output file with these results. Thus, this file can be used by decision models to choose services for compositions.



Algorithm 1: FSS algorithm.

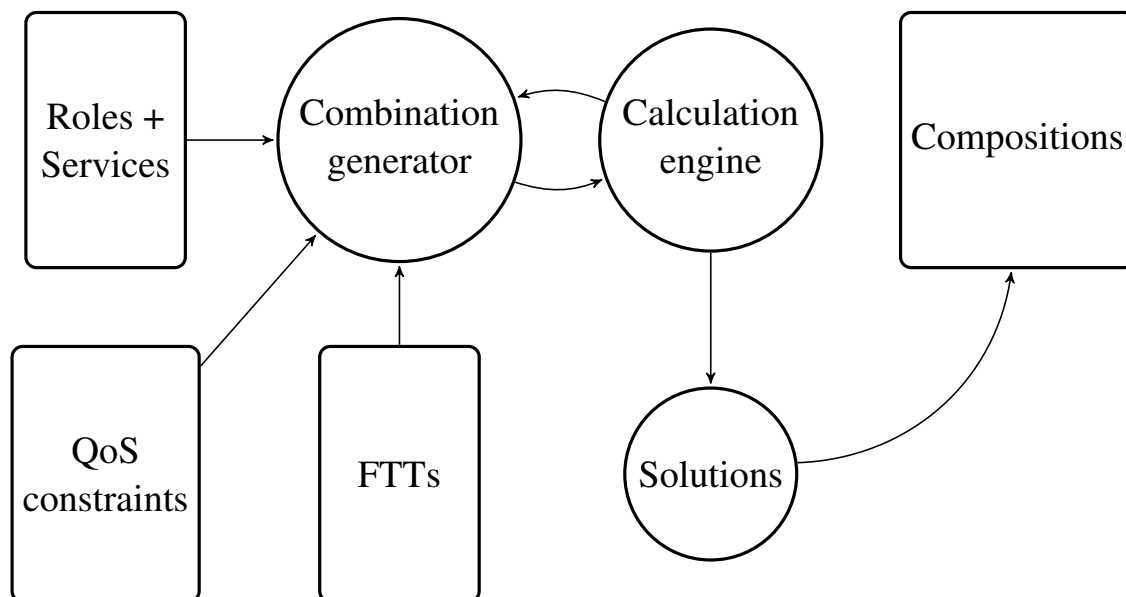


Figure 1. FSS process.

4.1. Calculation Engine

The formulas to calculate the expected QoS constraints for response time and reliability are presented in this subsection. We use these formulas to select combinations of services that reach these constraints according to the different characteristics of each FTT. These formulas are based in the work of Zheng and Lyu [Zheng and Lyu 2010]. Response time

must be less or equal than the expected value for the composition. Reliability must be greater or equal than the expected value for the composition. If both values are met the combination is selected.

The formulas represent the QoS values of each service used in a composition. Each FTT has a different strategy to form combinations of services. For the response time formulas, shown in Equations 1 to 4, the response time of each service used from all roles is summed to obtain the response time of a composition. The formulas presented in Equations 5 to 8 calculate the reliability of a whole composition considering each individual reliability value.

Equation 1 serves to evaluate the response time for Retry. For each service of a role i , its response time rt_i is multiplied by the number of tries ($tries_i$) that a service in i is requested. This value is summed for all services used in the n roles of the composition. The response time for RB is calculated using Equation 2. First, the response times rt_{im} of the requested equivalent services of a same role are summed. Next, the response times of all n roles are summed to obtain the response time of the composition.

The response time for NVP is shown in Equation 3. The maximum response time value among all equivalent services rt_{im} of a same role i is chosen. These values are summed for all n roles to obtain the total response time of a composition. As all services of each role are called in NVP, only the higher response time value of each role needs to be summed. Equation 4 is used for calculating response time for Active. The minimum response time value among all equivalent services rt_{im} of a same role i is chosen. This is because the fastest service of each role is used in Active. The response time values for all n roles are summed to obtain the total response time of a composition.

Equation 5 is used for calculating the reliability of a composition for Retry. First, the reliability of each service rel_i of a role i is raised to the power of the number of tries ($tries_i$). Then, the reliability of the entire composition is the product of the obtained values for all n roles. For RB, the Equation 6 calculates the composition reliability as the product of the requested equivalent services (rel_{im}) of a role i by the product of all n roles. For Retry and RB, only the services used of each role are considered.

Equation 7 represents the reliability for NVP. For each role, we consider that $\frac{m}{2} + 1$ services return an expected result. The formula is the product of these $\frac{m}{2} + 1$ equivalent services multiplied for all n roles. This estimation is different from that used by Zheng and Lyu [Zheng and Lyu 2010], which consider that the failure-rate of m services is $\frac{m}{2} + 1$. Finally, Equation 8 is used to calculate the reliability for Active. The product of all m equivalent services is multiplied for n roles to obtain the reliability of a composition. Thus, all services of each role are considered to obtain the reliability of a composition.

$$RT_{retry} = \sum_{i=1}^n (tries_i * rt_i) \quad (1)$$

$$RT_{rb} = \sum_{i=1}^n (rt_{i1} + rt_{i2} + \dots + rt_{im}) \quad (2)$$

$$RT_{nvp} = \sum_{i=1}^n \max(rt_{i1}, rt_{i2}, \dots, rt_{im}) \quad (3)$$

$$RT_{active} = \sum_{i=1}^n \min(rt_{i1}, rt_{i2}, \dots, rt_{im}) \quad (4)$$

$$Rel_{retry} = \prod_{i=1}^n (1 - (1 - rel_i)^{tries_i}) \quad (5)$$

$$Rel_{rb} = 1 - \prod_{i=1}^n ((1 - rel_{i1}) * (1 - rel_{i2}) * \dots * (1 - rel_{im})) \quad (6)$$

$$Rel_{nvp} = 1 - \prod_{i=1}^n ((1 - rel_{i1}) * (1 - rel_{i2}) * \dots * (1 - rel_{i[\frac{m}{2}]+1})) \quad (7)$$

$$Rel_{active} = 1 - \prod_{i=1}^n ((1 - rel_{i1}) * (1 - rel_{i2}) * \dots * (1 - rel_{im})) \quad (8)$$

4.2. Growth of Combinations

The different strategies implemented by each FTT determine the way to find possible solutions to the QoS constraints. Retry uses one service at a time for different roles. This fact leads to a certain number of combinations of services. Each combination is equivalent to a solution that represents a composition. The number of tries influences the parameter values and whether a combination reaches the expected values, but does not impact the growth of combinations. The growth of combinations for Retry is shown in Equation 9. The number of combinations is the product of the number of equivalent services ($size_i$) for all n roles.

$$Combinations_{retry} = \prod_{i=1}^n size_i \quad (9)$$

For RB, NVP, and Active, FSS makes all possible combinations of services. Compared with Retry, the number of combinations is exponentially greater. Equation 10 shows the growth of combinations for these FTTs. The number of equivalent services per role impacts the number of combinations more than the number of roles, as shown in Table 1.

$$Combinations_{rb,nvp,active} = \prod_{i=1}^n 2^{size_i} - 1 \quad (10)$$

5. Experimental Evaluation

Our experiments aimed at measuring the effectiveness of FTTSERVICESELECTOR to find solutions for the QoS constraints using the four FTTs. The effectiveness was measured by time spent by FSS for generating and calculating the combinations, and storing solutions. The goal is to evaluate the scalability of the technique according to the growth of combinations. The factors that affect the scalability of our service selection technique are: the number of roles, the number of equivalent services, the four FTTs, and the QoS constraints. FSS is a multidimensional multiple-choice knapsack problem (MMKP), which is known as a NP-hard problem [Kumar 1992].

We used yaml files as data input to simulate compositions. These files contain the available services for each role. Each service is composed of fixed QoS values for

response time and reliability. We chose different sizes of compositions to the experiments. These sizes varied from three to ten roles and from three to ten equivalent services for each role. We used the same number of equivalent services for the roles in a same file. For Retry, we used a number of three tries for each service. We fixed an arbitrary value for the number of tries because it does not influences the growth of combinations.

The expected constraint values for response time and reliability for the composition were fixed in 5,000 seconds and 0.50, respectively. We used these values to store all combinations as solutions (i.e., all compositions meet the expected QoS constraints) . Thus, we avoid that variations in the number of stored solutions influenced the execution time of FSS.

The times to create and evaluate all combinations were measured as well as the number of combinations generated. We consider that each role is called sequentially from the first to the last role.

We implemented FSS in Java using JDK 1.7.0_45. The experiments were performed using a 64-bit PC with processor Intel Core i7 model 2620M and 8GB of RAM. The OS used was an Ubuntu 12.10. The experiments were repeated ten times for each composition size and the averages of time spent between these ten executions were recorded in the results. Since the dispersion of the results was low, this information is not presented in the graphs to improve the visualization.

All the code, data, and documentation used in the experiments is publicly available on GitHub². This information is provided to facilitate the reproducibility of all the experiments reported in this paper.

6. Results

The size of combinations generated for different roles and equivalent services is shown in Table 1 (columns Retry and RB, NVP, Active). The values were separated for Retry and for the other FTTs, since the other FTTs have the same number of combinations. The column ES represents the number of equivalent services per role.

In Table 1, it is possible to see the exponential growth of the combinations generated by FTTs. The larger the number of roles, the larger the number of combinations. In the same way, the number of combinations grows when the number of equivalent services per role grows.

For RB, NVP, and Active, it can be noticed that the number of equivalent services per role has a great impact in the growth of the combinations than the number of roles. For three roles and ten equivalent services, more than 1 billion combinations were generated. Using ten roles and three equivalent services per role, the number of combinations was around 280 million. For Retry, the growth of combinations was more influenced by the number of tries and the number of roles, since only one service was used per role.

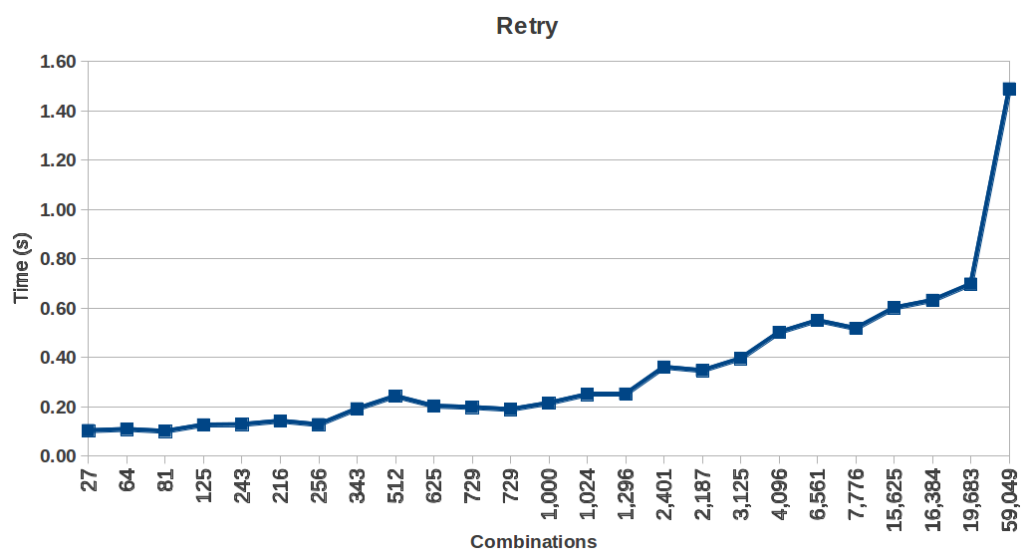
Figure 2 shows the relation between the time spent and the number of combinations obtained by FSS for Retry. The time includes generation, calculation, and recording of the solutions. The x axis is the number of combinations. The y axis represents the time spent in seconds. As the number of combinations for Retry is smaller compared

²<https://github.com/higoramario/ServiceSelector>

Table 1. Number of combinations generated.

Roles	ES	Retry	RB, NVP, Active	Roles	ES	Retry	RB, NVP, Active
3	3	27	343	5	3	243	16,807
3	4	64	3,375	5	4	1,024	759,375
3	5	125	29,791	5	5	3,125	28,629,151
3	6	216	250,047	5	6	7,776	992,436,543
3	7	343	2,048,383	6	3	729	117,649
3	8	512	16,581,375	6	4	4,096	11,390,625
3	9	729	133,432,831	6	5	15,625	887,503,681
3	10	1,000	1,070,599,167	7	3	2,187	823,543
4	3	81	2,401	7	4	16,384	170,859,375
4	4	256	50,625	8	3	6,561	5,764,801
4	5	625	923,521	9	3	19,683	40,353,607
4	6	1,296	15,752,961	10	3	59,049	282,475,249
4	7	2,401	260,144,641	–	–	–	–

to the other FTTs, the time spent by FSS is less than 0.8 seconds, except for one case. For ten roles and three equivalent services per role, which generated around 59 thousand combinations, FSS spent around 1.5 seconds.

**Figure 2. Time spent x Combinations for Retry.**

The results of Retry, RB, NVP, and Active were separated in two figures to highlight the differences obtained by the FTTs for the number of combinations. Figure 3 shows the time spent in the y axis. The x axis is the number of combinations (in thousands) up to two million combinations. Figure 4 shows the number of combinations to values greater than two million combinations.

For the number of combinations in Figure 3, the time spent by RB, NVP, and Active were similar. NVP spent less time to be executed by FSS than RB and Active with a small difference. In Figure 4, as the number of combinations grew, the difference

in time spent between NVP related to RB and Active also grew. For up to 40 million combinations, FSS generated solutions for all FTTs spending no more than 16 minutes. This is equivalent of nine roles and three equivalent services. For Retry the time spent was smaller than RB, NVP, and Active due to the number of combinations.

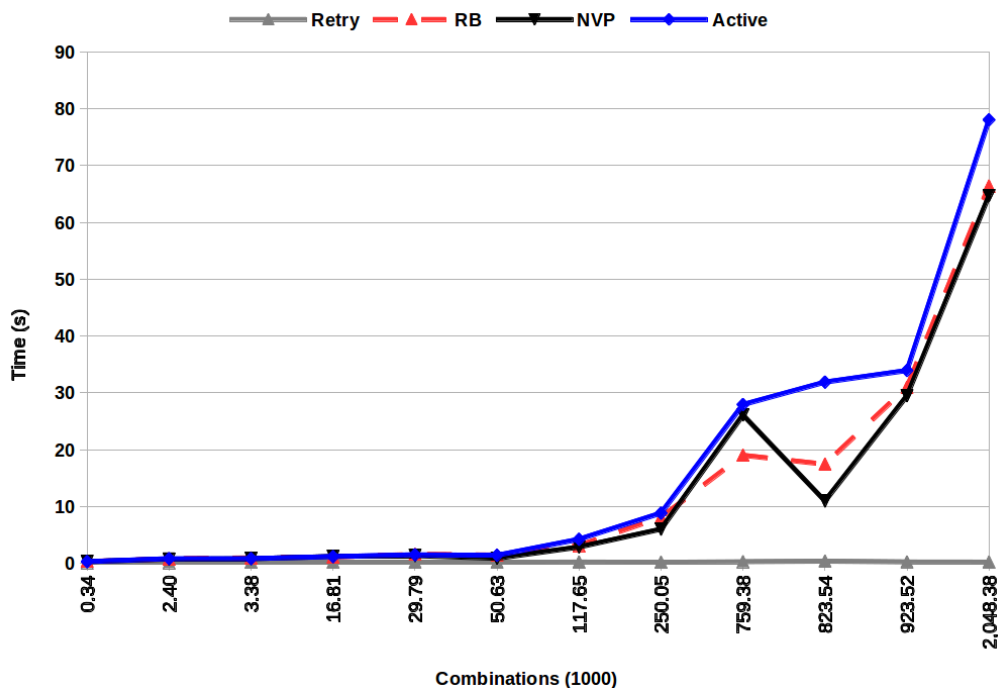


Figure 3. Time spent x Combinations for Retry, RB, NVP and Active (Part 1).

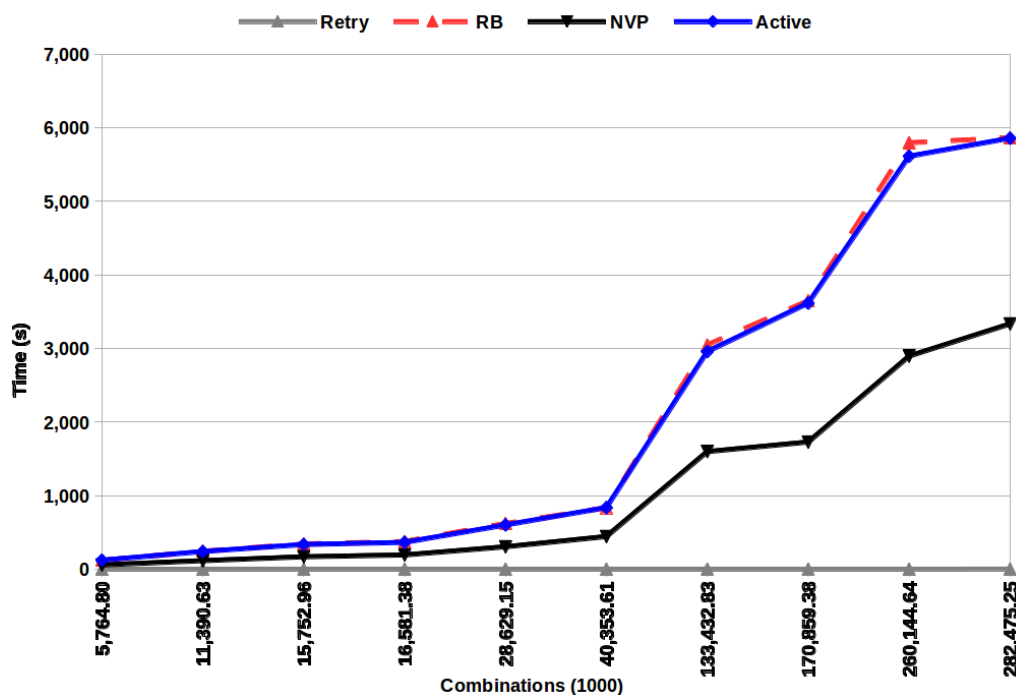


Figure 4. Time spent x Combinations for Retry, RB, NVP and Active (Part 2).

7. Discussion

The results show that FSS is scalable for compositions up to ten roles and up to ten equivalent services. The time spent by FSS to obtain solutions increases exponentially with the number of equivalent services per role. For most of the cases, FSS spent less than 60 minutes to generate the solutions. In the worst case for the values used in the experiments, FSS generated solutions in less than 140 minutes. Although these times can be viewed as excessive, they are irrelevant if FSS is used in a system where the QoS requirements of the services are monitored and stored in a historical way. In this scenario the mechanism represents a valuable contribution, in which FSS uses these previous information to the service selection process. Besides, if some services are frequently used in conjunction, due for example to delay or other network limitations, they can be viewed as only one service, which can reduce the number of services considered to each composition.

Retry can generate solutions with a complexity much smaller than the other FTTs, but the strategy considers only one equivalent service at a time per role. As the size of combinations increases, the execution time also increases, specially for RB, NVP, and Active. The time obtained for RB and Active are similar for most of the cases. These two FTTs spent more time than NVP to generate the solutions. As the number of combinations is the same for RB, NVP, and Active, we realize that the difference occurs due to the calculation of reliability for the FTTs. For NVP, FSS calculates the reliability for half of the equivalent services when compared to RB and Active.

The time spent by FSS is directly related to the FTT strategies. If the FTT needs to generate a large number of combinations, the time spent could grow proportionally. In the same way, the more the number of roles and equivalent services, the more the time spent to execute FSS. The number of solutions found by the techniques depends on the QoS constraints values passed as parameters and the formulas of each FTT. FSS could find the solutions that met these constraints even for the highest number of roles and services evaluated.

One problem faced during the experiments was the size of data generated to record the solutions calculated by FSS. The larger the number of combinations that meet the QoS constraints, the higher the execution time to run FSS. It occurs due to problems with memory space used by the JVM and the management of the Java Garbage Collector. When we use QoS values that were met by a reduced number of solutions (with up to five millions solutions), the execution of FSS did not present these problems.

The use of the CSP backtracking approach to service selection seems to be suitable when we are interested in finding solutions for generating all possible combinations between equivalent services through roles of compositions. Thus, considering that variations in the QoS values of services in a composition may occur, it is possible to select different compositions to maintain the expected QoS constraints.

8. Threats to Validity

Our experiments were performed using files with simulated QoS constraint values for the services and compositions. This approach is executed using static information from services. The use of values obtained from the execution of real compositions could lead to different results. The use of files with fixed values does not consider dynamic variations

that could occur during executions of compositions such as network connections or execution time of services providers. Other issues that can influence the results are the use of FSS in different domains and the use of other QoS requirements.

During the experiments, we did not vary the number of equivalent services per role. Thus, experiments using more scenarios could lead to different results. Another threat is the number of solutions. Depending on the constraint values, the number of solutions in memory could affect the time spent to generate all solutions. We avoid this threat using QoS constraints so that all combinations generated were accepted as solutions.

9. Conclusions and Future Work

FTTServiceSelector (FSS) is a technique that uses a CSP backtracking approach to select services for compositions. FSS uses four Fault-Tolerance Techniques to compose different strategies for the selection process: Retry, Recovery Block, N-Version Programming, and Active Replication. The selection is made using two QoS constraint values for compositions: response time and reliability.

Experiments with different composition sizes were performed to evaluate the effectiveness of FSS to find solutions using QoS constraints. The efficiency of FSS was measured evaluating the execution time to calculate the solutions. The results showed that FSS could be used to select services that meet the QoS constraints with times between 2 seconds to 140 minutes for compositions varying from three roles and three equivalent services per role to ten roles and three equivalent services per role. For most cases FSS generated solutions spending no more than 17 minutes. The information generated by FSS could be used by decision models responsible for selecting services for compositions in automated processes.

As future work, we intend to make improvements to reduce the time spent to generate the solutions. One possible way to improve the technique could be the use of filters to exclude individual services that can degrade the composition. The use of other QoS requirements and FTTs are other issues to be addressed in the future. Moreover, the use of historical data about QoS values obtained from previous executions of compositions can be used to make experiments in the future.

We also intend to use other CSP strategies, such as local search algorithms, to measure the feasibility to generate solutions reducing the time spent. For instance, if we are interested in finding only one solution to a composition, other CSP strategies, such as Constraint Propagation, could be evaluated in the future to obtain more knowledge about the use of CSP for selecting services. Experiments in runtime environments using more roles and more equivalent services are also among our future work.

References

- Ai, L. and Tang, M. (2008). A penalty-based genetic algorithm for qos-aware web service composition with inter-service dependencies and conflicts. In *Proceedings of the 2008 International Conference on Computational Intelligence for Modelling Control Automation*, pages 738–743.
- Alrifai, M., Risse, T., and Nejdl, W. (2012). A hybrid approach for efficient web service composition with end-to-end qos constraints. *ACM Transactions on the Web*, 6(2):1–31.

- Ammann, P. and Knight, J. (1988). Data diversity: an approach to software fault tolerance. *IEEE Transactions on Computers*, 37(4):418–425.
- Avizienis, A. (1995). *The Methodology of N-Version Programming*, chapter 2, pages 23–42. John Wiley & Sons Ltd. Software Fault Tolerance.
- Avizienis, A., Laprie, J.-C., Randell, B., and Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33.
- Chereque, M., Powell, D., Reynier, P., Richier, J.-L., and Voiron, J. (1992). Active replication in delta-4. In *22th International Symposium on Fault-Tolerant Computing. FTCS-22. Digest of Papers.*, pages 28–37.
- Davis, E. (1987). Constraint propagation with interval labels. *Artificial Intelligence*, 32(2):281–331.
- Dobson, G. (2006). Using ws-bpel to implement software fault tolerance for web services. In *32nd EUROMICRO Conference on Software Engineering and Advanced Applications*, pages 126–133.
- Jussien, N. and Lhomme, O. (2002). Local search with constraint propagation and conflict-based heuristics. *Artificial Intelligence*, 139(1):21–45.
- Kattepur, A., Georgantas, N., and Issarny, V. (2013). Qos composition and analysis in reconfigurable web services choreographies. In *International Conference on Web Services*, Santa Clara, California, USA.
- Kumar, V. (1992). Algorithms for constraint-satisfaction problems: A survey. *AI magazine*, 13(1):32.
- Menasce, D. (2002). Qos issues in web services. *IEEE Internet Computing*, 6(6):72–75.
- Papazoglou, M. (2003). Service-oriented computing: concepts, characteristics and directions. In *Fourth International Conference on Web Information Systems Engineering*, pages 3–12.
- Papazoglou, M., Traverso, P., Dustdar, S., and Leymann, F. (2007). Service-oriented computing: State of the art and research challenges. *Computer*, 40(11):38–45.
- Peltz, C. (2003). Web services orchestration and choreography. *Computer*, 36(10):46–52.
- Randell, B. and Xu, J. (1995). *The Evolution of Recovery Block Concept*, chapter 1, pages 1–17. John Wiley & Sons Ltd. Software Fault Tolerance.
- Rao, J. and Su, X. (2005). A survey of automated web service composition methods. In Cardoso, J. and Sheth, A., editors, *LNCS on Semantic Web Services and Web Process Composition*, volume 3387, pages 43–54. Springer Berlin Heidelberg.
- Salatge, N. and Fabre, J. C. (2007). Fault tolerance connectors for unreliable web services. In *International Conference on Dependable Systems and Networks*, pages 51–60.
- Yu, T. and Lin, K.-J. (2005). Service selection algorithms for web services with end-to-end qos constraints. *Information Systems and e-Business Management*, 3(2):103–126.
- Zheng, Z. and Lyu, M. (2010). An adaptive qos-aware fault tolerance strategy for web services. *Empirical Software Engineering*, 15(4):323–345.